EmbedSim GUI · Block Diagram Designer

Student Guide · Fast Read

Abstract

A visual block-diagram tool for EmbedSim. Drag blocks, wire them, generate a runnable Python simulation file — no hand-coding of connections required. This guide covers the >> syntax, drag-and-drop canvas, code generation, MVC architecture, block registry, web extension, and thesis improvements.

1. WHAT THE GUI IS AND WHY IT EXISTS

Writing EmbedSim simulation code by hand becomes a transcription marathon for a PMSM motor controller with twelve blocks, seventeen wiring connections, and five parameters each. A mis-typed variable name gives a runtime exception, not a compile error.

The GUI eliminates this class of error entirely. You assemble the simulation visually — dragging blocks, connecting pins — and the tool generates the Python file for you. The generated file is syntactically correct by construction.

The current prototype (embedsim_gui.py) implements:

- A block palette with all registered EmbedSim block types, organised by category
- A node-editor canvas where blocks become draggable nodes with input/output pins
- Pin-to-pin wiring by dragging from an output pin to an input pin
- A parameter sidebar — click any node to edit its parameters inline
- An algebraic loop validator that runs a DFS cycle-detection algorithm
- A code generator that produces a complete, runnable EmbedSim .py file

What it does NOT yet do:

- Run the simulation from within the GUI — you must open a terminal
- Display plots inside the application
- Auto-discover new blocks — adding a block requires a manual entry in BLOCK_REGISTRY
- Persist the canvas to a file (no save/load of the diagram itself)
- Run in a web browser — it requires a desktop Python installation with Dear PyGui

2. THE >> SYNTAX — EMBEDSIM'S WIRING OPERATOR

The >> symbol means: "connect the output of the left block to an input of the right block." When Python evaluates sin_src >> gain, it calls sin_src.__rshift__(gain). EmbedSim's VectorBlock base class defines

this method to register the connection in the simulation's adjacency graph. No actual data flows at this point — you are only declaring the topology.

Chaining works: sin_src >> summer >> gain >> sink. This gives simulations the readability of a schematic diagram — the key design decision that makes EmbedSim files both human-writable and machine-parseable, which is precisely what enables the GUI's code generator.

Multi-input blocks: VectorSum with n_inputs=2 expects two upstream connections. The order in which you write them matters — the first >> connection is input 0, the second is input 1. The GUI preserves connection order.

Algebraic loops occur when a signal feeds back into itself without any delay. The GUI's Validate button runs a depth-first search on the connection graph. Any cycle not passing through a VectorDelay is flagged as an algebraic loop with the names of the involved blocks.

3. DRAG-AND-DROP BLOCK PLACEMENT

The GUI uses Dear PyGui's node editor widget. Each EmbedSim block becomes a node with input/output pins (attributes). Think of it as a physical patch bay — devices have input and output jacks, and you connect them by dragging a cable from output to input.

Double-clicking the palette fires a callback that creates a NodeInst with uid, var_name, and params, then calls dpg.node() to place it on the canvas. Position is computed from the current node count, arranged in a 4-column grid so blocks do not pile up.

When you draw a wire, Dear PyGui fires cb_link_created with (attr_out_tag, attr_in_tag). This is stored in the links dictionary. The attr_to_node dictionary maps each pin tag back to the block's node ID — together they give the complete signal flow graph.

Clicking a node opens the sidebar. Widgets adapt to parameter type: float, int, bool, or string. Each widget's callback calls _update_param() which updates NodeInst.params and refreshes the canvas label.

4. CODE GENERATION — FROM CANVAS TO PYTHON FILE

The generate_code() function is a single-pass string builder. It walks the nodes, links, and attr_to_node dictionaries and emits a complete Python file in five stages:

- Stage 1 — Collect import statements from each block's BlockDef
- Stage 2 — Emit simulation parameters (T_SIM, DT)
- Stage 3 — Emit block instantiation lines (e.g. sin_src = SinusoidalGenerator('sin_src', amplitude=1, freq=50))
- Stage 4 — Emit wiring lines (e.g. sin_src >> gain)
- Stage 5 — Emit EmbedSim(), scope.add(), sim.run(), and plot calls

The wiring stage iterates links.values() and converts each (attr_out_tag, attr_in_tag) pair back to variable names via attr_to_node and nodes. An emitted set prevents duplicate lines when a block has multiple outputs going to the same target.

5. RUNNING THE SIMULATION

Currently: generate and save the .py file from the GUI, then run it in a terminal. The GUI does not execute any simulation code itself. This breaks the creative flow — every gain change requires save, switch to terminal, run, wait, observe, close, switch back.

What your thesis should add — an embedded Run button:

- Generate code to a temporary string (already possible with generate_code())
- Execute it in a background thread — never in the GUI thread, which would freeze the interface
- Capture the sim.scope data
- Render the scope data as plots embedded in the GUI

Dear PyGui supports texture-based image rendering. A Matplotlib figure can be converted to a NumPy RGBA array and pushed to a Dear PyGui texture, appearing as an inline image in the GUI window.

6. ARCHITECTURE — MODEL · VIEW · CONTROLLER

The prototype works, but the model (data), view (GUI), and controller (callbacks) are mixed in one 700-line file. A clean MVC separation is not just academic tidiness — it is the prerequisite for the web version (Section 8), where the Model and Controller must run identically whether the View is Dear PyGui or a browser.

Model layer (model.py): DiagramModel, NodeInst, BlockDef, generate_code(), validate(). No dpg.* calls. No HTML/JS. Fully testable without any GUI.

View layer: responsible only for displaying the model and forwarding user events to the controller. No direct model mutation. No business logic. No loop detection.

Controller layer: receives user events, updates the model, instructs the view to refresh. Depends only on the Model and View interfaces — the same controller works for both Dear PyGui and the web view.

7. THE BLOCK REGISTRY — ADDING NEW BLOCKS

In the prototype, every block must be manually added to BLOCK_REGISTRY. Adding a new EmbedSim block class requires a manual edit. For a framework with 30+ block types this is error-prone.

Python's inspect module can walk an installed package and find all subclasses of VectorBlock automatically via pkgutil.walk_packages(). For richer registration (colour, category, description), block classes can carry metadata as class attributes (TOPO_CATEGORY, TOPO_COLOR, description). Auto-discovery reads these if present, with sensible defaults if not.

After this one-time addition per block class, any new block added to EmbedSim appears automatically in the GUI palette on the next start — no changes to the GUI code required.

8. THE WEB VERSION — DESIGN AND FEASIBILITY

Dear PyGui requires a desktop Python installation — not available on mobile devices, Chromebooks, or locked-down corporate environments. A web version makes EmbedSim accessible to IoT engineering students on any device with a browser.

Architecture: Python simulation code runs on a server (FastAPI + uvicorn). The browser provides the visual interface. They communicate over WebSockets with JSON messages (add_node, add_link, update_param, validate, run, run_progress, run_complete).

Canvas library choices:

- Drawflow (MIT) — built-in node-editor, lightweight. Best for a thesis prototype
- React Flow (MIT) — excellent, React-based. Best if you use React
- jsPlumb (MIT) — good, older API. Acceptable alternative

A minimum viable web demo: ~200 lines of Python (server + controller) and ~150 lines of HTML/JavaScript (frontend). Achievable in one week once the MVC refactor is done.

9. THESIS IMPROVEMENTS — WHAT TO BUILD AND WHY

A) MVC Refactor (foundational, must do) — prerequisite for all other improvements. Document with before/after line-count comparison and a test showing identical code generation regardless of view.

B) Diagram Persistence (save/load JSON) — DiagramModel.to_dict()/from_dict(). Enables version control of diagrams alongside the generated .py file.

C) Auto-Discovery of blocks — implement discover_blocks(). Measure: GUI code changes when a new block is added. Before: 10+ lines. After: 0 lines.

D) Integrated Run + Inline Plots — background simulation runner. Edit-run-observe cycle drops from 6 manual steps to 1 button click.

E) Web Version (FastAPI + Drawflow) — highest reach. Any device, any browser. Reuses DiagramModel unchanged from the desktop version.

10. DELIVERABLES AND THESIS CLAIMS

Files to deliver:

- embedsim_gui/model.py — DiagramModel, NodeInst, BlockDef (no GUI imports)
- embedsim_gui/view_dpg.py — DearPyGuiView (all dpg.* calls isolated here)
- embedsim_gui/controller.py — DiagramController (handles all user events)
- embedsim_gui/discovery.py — discover_blocks() via inspect
- embedsim_web_server.py — FastAPI + WebSocket server, reuses DiagramModel
- static/index.html + app.js — Drawflow canvas + JSON protocol
- tests/test_model.py — unit tests: add_node, add_link, validate, generate_code
- user_study/results.csv — task-completion times: prototype vs improved GUI

Thesis claims: (1) MVC separation — DiagramModel has zero imports of Dear PyGui or any web framework; generates identical code from desktop or web view. (2) Extensibility — adding a new block requires zero GUI code changes. (3) Web accessibility — full workflow runs in a browser with no local Python installation.